

Robot Hiwonder JetMax

Bras manipulateur articulé intelligent sous NVIDIA Jetson Nano et ROS

Projet de Robotique Appliquée

Arts et Métiers Campus Rabat

Année universitaire 2025-2026

1. Mise en route

- ➊ **Getting Ready**
Kits, zones de travail, démarrage
- ➋ **Présentation du Robot**
Architecture, actionneurs, communication
- ➌ **Prise en Main Rapide**
APP WonderAi, calibration, Lab
- ➍ **Contrôle PC & Actions**
Logiciel web, éditeur de séquences

2. Fondements Techniques

- ➎ **Modélisation Cinématique**
DH, MGD, MGI, singularités
- ➏ **Configuration Jetson Nano**
OS, CUDA/cuDNN, validation
- ➐ **Framework ROS**
Nœuds, topics, services
- ➑ **Trajectoires & PID**
Planification, asservissement

3. Applications IA & Outils

- ➒ **Vision Artificielle**
HSV, homographie, Pick-and-Place
- ➓ **Jeux de Vision IA**
YOLOv5, MTCNN, GoogLeNet, Lab
- ➔ **Outils Terminaux**
Stylo, grande pince, ventouse EM

4. Enseignements Avancés

- ➕ **Cours de Prog. Avancée**
Jetson, Extension Board, Capteurs
- ➖ **Bases d'OpenCV**
Pixels, espace Lab, Contours, Suivi
- ➗ **Deep Learning & YOLOv5**
Orthographe, Masques, TM1640

Trois niveaux de kits

Kit	Contenu principal	Usage
Starter	Bras + servos	Cinématique de base
Standard	+ Ventouse + Caméra	Vision IA
Advanced	+ Pince + Stylo + Aimant	Applications complètes

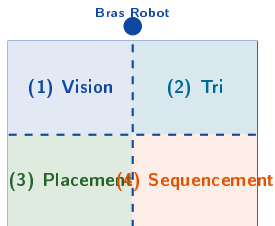
- **Jetson Nano B01** embarqué (GPU Maxwell 128 cœurs)
- **Caméra HD grand angle 120°** — interface USB
- **Carte d'extension JetMax** (MCU STM32 dédié)
- Servomoteurs **HTS-35H** — engrenages métalliques

Outils terminaux disponibles

- 1 **Ventouse à air** (défaut)
Pression négative, objets plats
- 2 **Grande pince mécanique**
Classification fruits/légumes (GoogLeNet)
- 3 **Ventouse électromagnétique**
Pièces métalliques (vis, écrous)
- 4 **Porte-stylo**
Dessin, écriture, émotions
- 5 **Petite pince**
Objets fins

4 zones du tapis officiel

- ❶ **Zone (1) Reconnaissance visuelle**
Surface de dépôt des objets à analyser par la caméra
- ❷ **Zone (2) Tri**
Cases Rouge / Vert / Bleu + cartes déchets recyclables
- ❸ **Zone (3) Placement robot**
Destination de dépose calibrée (coordonnées fixes)
- ❹ **Zone (4) Séquencement**
Cartes alphabet / chiffres pour l'IA de séquencement



Le tapis imprimé fourni avec le kit définit les repères physiques utilisés lors de la calibration par homographie.

Procédure de démarrage

- 1 Connecter l'alimentation **DC 12 V / 5 A**
- 2 Basculer l'interrupteur en position **ON**
- 3 Attendre **3 bips sonores** $\Rightarrow$ démarrage réussi
- 4 Le Jetson Nano lance Ubuntu et crée un **hotspot Wi-Fi HW-XXXXXXX** (mode AP)

Signification des 3 bips

Les 3 bips confirment que le firmware MCU (STM32) est chargé et que le bus série vers les servos HTS-35H est actif et prêt.

Modes de connexion réseau

- **Mode AP** (par défaut)
Hotspot autonome HW-XXXXXXX
IP fixe : 192.168.149.1
- **Mode Station (LAN)**
Connexion à un réseau Wi-Fi existant
IP dynamique attribuée par DHCP

Outils d'accès distant

- **SSH** : accès console sécurisé
- **NoMachine (NX)** : bureau graphique (RViz, débogage, éditeur visuel)
- **Navigateur** : <http://192.168.149.1>
Logiciel PC JetMax intégré

Installation et connexion

- **iOS** : App Store → WonderAi
- **Android** : APK officiel Hiwonder
- Connexion **directe** au hotspot
HW-XXXXXXX (mode AP)
ou via **réseau LAN** partagé

Calibration avec AprilTag

- 1 Placer le bloc **AprilTag** sur le tapis
- 2 L'APP détecte la balise optique et calcule la transformation pixel → repère physique
- 3 Valider : les coordonnées sont désormais référencées au repère du bras

5 modes de l'application

- 1 **Position Calibration**
Calibrage initial avec AprilTag
- 2 **Robot Control**
Pilotage manuel avec caméra live
- 3 **Waste Sorting**
Tri automatique de déchets (IA)
- 4 **Color Sorting**
Tri par couleur rouge/vert/bleu
- 5 **Block Stacking**
Empilement automatique de blocs

5 zones de l'interface

- 1 **Vue caméra live**
Flux vidéo temps réel HD
- 2 **Contrôle souris**
Cliquer sur l'image = cibler une position
- 3 **Contrôle cartésien**
Boutons $\pm X$, $\pm Y$, $\pm Z$
- 4 **Sliders articulaires**
Réglage individuel par ID de servo (1-4)
- 5 **Contrôle effecteur**
Ventouse ON/OFF · Pince ouvrir/fermer

Interaction vision directe

Le mode souris permet de **cliquer directement sur l'image caméra** pour envoyer le bras vers la position souhaitée — vision + cinématique inverse **transparente pour l'utilisateur**.

Flux d'utilisation typique

- 1 Se connecter au hotspot HW-XXXXXXXX
- 2 Ouvrir WonderAi → Robot Control
- 3 Calibrer avec le bloc AprilTag
- 4 Cliquer sur un objet dans la caméra
- 5 Le bras se déplace précisément vers la cible

L'espace de couleur Lab

- **L** : Luminosité (0 = noir, 255 = blanc)
- **a** : Axe Vert $\longleftrightarrow$ Rouge
- **b** : Axe Bleu $\longleftrightarrow$ Jaune

Avantage clé : la couleur (a , b) est **structurellement indépendante** de la luminosité (L), contrairement à RGB et HSV.

Procédure de calibration

- 1 Placer l'objet cible devant la caméra
- 2 Augmenter **a_min** jusqu'à ce que le masque devienne **blanc uniforme**
- 3 Ajuster **L** et **b** selon l'éclairage ambiant
- 4 Vérifier la robustesse en déplaçant l'objet

Pourquoi pas HSV ?

En HSV, la Teinte est stable mais la Saturation peut varier sur les bords de l'objet (reflets). En **Lab**, l'axe a capture directement le rouge pur et reste robuste sous des éclairages variés.

Seuils Lab typiques (rouge)

Canal	Min	Max
L (luminosité)	0	255
a (vert-rouge)	150	255
b (bleu-jaune)	0	255

- **Type** : Robot manipulateur articulé en série (structure anthropomorphe 3R + axe outil)
- **DDL actifs** : **4 degrés de liberté**
 - ① Base (θ_1) — rotation azimutale (240°)
 - ② Épaule (θ_2) — extension verticale
 - ③ Coude (θ_3) — inclinaison avant-bras
 - ④ Effecteur (θ_4) — orientation de l'outil
- **Matériau** : Alliage d'aluminium anodisé (légèreté + rigidité, réduction des vibrations)

Spécifications opérationnelles

Portée maximale	289 mm
Couverture angulaire	240°
Charge utile (payload)	450 g
Répétabilité	± 1 mm
Poids total	≈ 1.6 kg

Outils terminaux interchangeables

Ventouse électromagnétique (défaut)
Pince de préhension mécanique · Porte-stylo

Servomoteurs HTS-35H

- Engrenages métalliques (zéro jeu mécanique)
- Couple : 15 kg·cm sous 7.4 V
- Bus série half-duplex 115 200 bps (daisy-chain)
- Rétroaction temps réel : position, température, tension

Capteurs

- **Caméra HD** 120° — *Eye-in-Hand*, 1280 × 720 px, interface USB
- **Ultrasonique lumineux** — 2–400 cm, LED RGB programmables (I2C/GPIO)

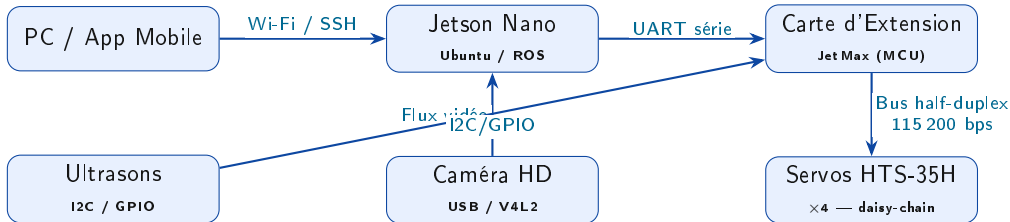
NVIDIA Jetson Nano (B01)

- CPU : Quad-core ARM Cortex-A57 @ 1.43 GHz
- GPU : Maxwell 128 cœurs CUDA
- RAM : 4 GB LPDDR4 partagée
- OS : Ubuntu 18.04 + SDK JetPack

Alimentation & Carte d'extension JetMax

- Entrée : DC 12 V / 5 A (60 W)
- Tensions dérivées : 7.4 V servos, 5 V Jetson, 3.3 V capteurs
- MCU dédié (STM32) — contrôle bas niveau

Architecture de Communication



Flux de traitement : Vision → Planification (MGI) → Exécution (servos)

Paramètres de Denavit-Hartenberg (DH)

Liaisons physiques :

- $L_1 = 92.54$ mm Base $\rightarrow$ Épaule
- $L_2 = 128.40$ mm Épaule $\rightarrow$ Coude
- $L_3 = 144.80$ mm Coude $\rightarrow$ Effecteur

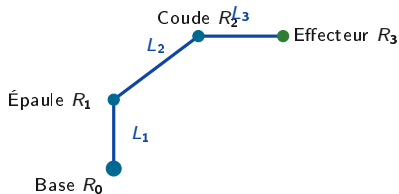


Table DH Standard

i	θ_i	d_i	a_i	α_i
1	θ_1	L_1 (92.54)	0	90°
2	θ_2	0	L_2 (128.40)	0
3	θ_3	0	L_3 (144.80)	0

Repères articulaires :

- R_0 : origine base, Z_0 vertical
- R_1 : axe épaule, Z_1 horizontal
- R_2 : pivot coude, $Z_2 \parallel Z_1$
- R_3 : extrémité outil, $Z_3 \parallel Z_2$

Matrice générale de passage du repère $i-1$ au repère i :

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 1 ($\alpha_1 = 90^\circ$, $a_1 = 0$)

$$T_1^0 = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joints 2 et 3 ($\alpha_i = 0$)

$$T_i^{i-1} = \begin{bmatrix} c_i & -s_i & 0 & L_i c_i \\ s_i & c_i & 0 & L_i s_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$c_i = \cos \theta_i, \quad s_i = \sin \theta_i$$

Produit matriciel global :

$$T_3^0 = T_1^0 \cdot T_2^1 \cdot T_3^2$$

Équations cartésiennes de position de l'effecteur

$$x = \cos \theta_1 \cdot (L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)) \quad (1)$$

$$y = \sin \theta_1 \cdot (L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)) \quad (2)$$

$$z = L_1 + L_2 \sin \theta_2 + L_3 \sin(\theta_2 + \theta_3) \quad (3)$$

Portée radiale plane : $r = L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)$, d'où $x = r \cos \theta_1$ et $y = r \sin \theta_1$.

Calcul de $(\theta_1, \theta_2, \theta_3)$ pour atteindre la cible (x, y, z) :

Étape 1 — Angle de la base

$$\theta_1 = \text{atan2}(y, x)$$

Étape 2 — Angle du coude (configuration *elbow-up*)

$$r = \sqrt{x^2 + y^2}, \quad z' = z - L_1, \quad D = \frac{r^2 + z'^2 - L_2^2 - L_3^2}{2L_2L_3} \quad \theta_3 = \text{atan2}(\sqrt{1 - D^2}, D)$$

Étape 3 — Angle de l'épaule

$$\theta_2 = \text{atan2}(z', r) - \text{atan2}(L_3 \sin \theta_3, L_2 + L_3 \cos \theta_3)$$

Zones singulières

- **Extension maximale** : $\theta_3 = 0$
portée maximale $r = 273.20$ mm
- **Singularité de base** : $x = y = 0$
outil aligné sur l'axe Z_0 vertical

Validation Python (cinematique_sim.py)

- Tests sur configurations représentatives
- Erreur de reconstruction MGD ◦ MGI :

$$\varepsilon \approx 10^{-15} \text{ mm}$$

- Seuil physique : ± 1 mm
- Visualisation 3D Matplotlib dynamique

Conclusion : Le modèle est valide — précision numérique bien supérieure à la tolérance physique.

OS et SDK

- Ubuntu 18.04 LTS Bionic Beaver
- NVIDIA JetPack (image système officielle)
- Python 3.6 (langage principal d'implémentation)

Accélération GPU (Maxwell 128 cœurs)

- **CUDA Toolkit 10.2** — compilation code GPU
- **cuDNN 8.2** — primitives Deep Learning
- **TensorRT** — inférence réseau de neurones temps réel

Vision par ordinateur

OpenCV 4.5.0 compilée avec support CUDA
Traitement vidéo haute cadence sans surcharge CPU

Connectivité réseau

- **Mode AP** : hotspot autonome
JetMax_XXXX / IP 192.168.149.1
- **Mode Station** : réseau local (LAN)
- **SSH** : accès console sécurisé
- **NoMachine (NX)** : bureau distant (RViz, debug)

Communication Série UART

- Port : `/dev/ttyTHS1` @ 115 200 bps
- Validation : trames binaires envoyées aux servos via bibliothèque Python `serial`

GPIO 40 broches

- Bibliothèque `Jetson.GPIO`
- Validation : pilotage broche 12 (buzzer) pour vérification des signaux numériques

Caméra USB

- Pilote V4L2 — `/dev/video0`
- Script : `cv2.VideoCapture(0)`
- Capture + écriture disque validées

Bilan — Système prêt

- ✓ Chaîne cinématique (servos) opérationnelle
- ✓ Flux vidéo haute résolution
- ✓ GPIO et UART fonctionnels
- ⇒ **Prêt pour ROS et Vision IA**

Qu'est-ce que ROS ?

Middleware standardisé pour robots : gestion du matériel, passage de messages inter-processus, gestion des packages.

Version ciblée : ROS Noetic / Ubuntu 20.04

Avantages clés :

- Architecture modulaire (nœuds indépendants)
- Réutilisation d'outils existants (RViz, Gazebo)
- Abstraction de la couche matérielle

Workspace Catkin :

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws && catkin_make
source ~/catkin_ws/devel/setup.bash
```

3 Packages JetMax

- 1 `jetmax_description`
URDF : géométrie, masses, inerties
- 2 `jetmax_control`
Interface série + services cinématiques
- 3 `jetmax_gazebo`
Simulation physique (Gazebo)

Master + Nœuds

- **roscore** : registre central de nommage, démarre en premier
- **Nœuds** : processus indépendants (Python ou C++)

Services — Req/Res synchrone

Mécanisme **bloquant** bidirectionnel

```
[Client] ==> Requete (x,y,z)
          <== Reponse (t1,t2,t3)
```

Topics — Pub/Sub asynchrone

Canal unidirectionnel Publication/Abonnement

```
[Publisher] ==> /jetmax/joint_states
            ==> [Subscriber]
```

- Type : `sensor_msgs/JointState`
- Fréquence cible : 10 Hz

Services cinématiques JetMax

- `/forward_kinematics`
 $(x, y, z) \rightarrow (\theta_1, \theta_2, \theta_3)$
- `/inverse_kinematics`
 $(\theta_1, \theta_2, \theta_3) \rightarrow (x, y, z)$

Nœuds développés

- `joint_publisher.py`
Consignes sinusoïdales à 10 Hz
- `joint_subscriber.py`
Affichage angles reçus en degrés
- `kinematics_service.py`
Serveur MGD/MGI déporté

CLI de débogage

```
rostopic list
rostopic echo /jetmax/joint_states
rostopic hz   /jetmax/joint_states
rosservice list
rosservice call /jetmax_control/...
rqt_graph
```

`rqt_graph` affiche la topologie complète du réseau de nœuds actifs en temps réel.

Trajectoire Rectiligne

$$P(t) = A + s(t) \cdot (B - A), \quad s(t) \in [0, 1]$$

Trajectoire Circulaire

$$x(t) = x_C + R \cos(\omega t)$$

$$y(t) = y_C + R \sin(\omega t)$$

$$z(t) = z_0 \quad (\text{hauteur constante})$$

Espace articulaire vs cartésien

- **Articulaire** : trajectoire courbe, risque de collision, calcul simple
- **Cartésien (retenu)**
Discretisation + MGI point par point — l'outil suit rigoureusement le chemin physique

Chaque articulation dispose de son propre régulateur de position :

$$u(k) = K_p e(k) + K_i \sum_j e(j) \Delta t + K_d \frac{e(k) - e(k-1)}{\Delta t}$$

Rôle de chaque terme

- K_p : ralliement rapide (oscillations si trop grand)
- K_i : annule l'erreur statique, compense la gravité
- K_d : amortit, limite le dépassement

Anti-windup + saturation vitesse appliqués.

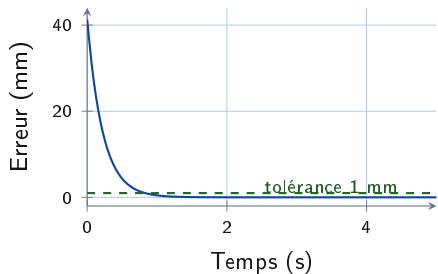
Paramètres de simulation

K_p	8.0
K_i	0.5
K_d	0.1
Δt	0.025 s (40 Hz)
Durée	5 s
τ_{moteur}	0.08 s

Performances mesurées

- Correction de 41.3 mm en < 1.2 s
- Erreur régime établi (cercle) :
 $\bar{e} = 0.015$ mm
- Erreur régime établi (ligne) :
 $\bar{e} = 0.018$ mm
- Erreur finale : 0.009 mm

Validation : Système stable et précis, compatible avec l'écriture et la manipulation par vision.



Espace de couleur HSV

- Conversion BGR $\rightarrow$ HSV
- **Hue** : couleur robuste aux variations d'éclairage

$$M(u, v) = \begin{cases} 1 & H \in [H_{\min}, H_{\max}] \wedge \\ & S \in [S_{\min}, S_{\max}] \wedge \\ & V \in [V_{\min}, V_{\max}] \\ 0 & \text{sinon} \end{cases}$$

Extraction du centroïde

- ❶ Filtrage morphologique (érosion + dilatation)
- ❷ Détection des contours (OpenCV)
- ❸ Moments d'image :

$$m_{pq} = \sum_{u,v} u^p v^q M(u, v)$$

$$u_c = \frac{m_{10}}{m_{00}}, \quad v_c = \frac{m_{01}}{m_{00}}$$

Zone de travail plane (z_{table} fixe) $\Rightarrow$ **homographie plane** H (3×3) :

$$\begin{bmatrix} x_w \\ y_w \\ w \end{bmatrix} = H \cdot \begin{bmatrix} u_c \\ v_c \\ 1 \end{bmatrix} \implies x_r = \frac{x_w}{w}, \quad y_r = \frac{y_w}{w}$$

Calibration

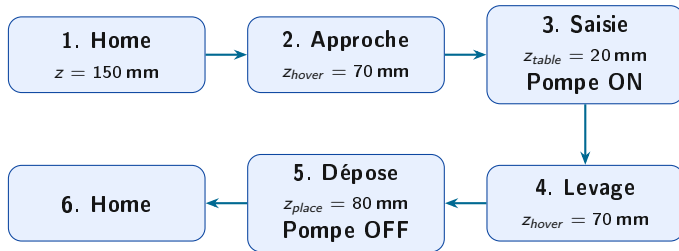
- 1 Mesure de **4 points de repère**
(pixel $\leftrightarrow$ coordonnées physiques)
- 2 Calcul de H via `cv2.findHomography`

Exemple de mapping

Détection pixel	(260, 205) px
Coord. robot	$X_r = 177.30$ mm
	$Y_r = 35.00$ mm



Séquence de Pick-and-Place



- $z_{\text{hover}} = 70 \text{ mm}$: évite les collisions latérales
- $z_{\text{table}} = 20 \text{ mm}$: descente + activation pompe
- Panier de dépose : (120, -120, 80) mm

Résultats vision_sim.py

- ✓ Détection bloc rouge sur fond vert
- ✓ Mapping pixel → physique
- ✓ MGI validé sur toutes les poses
- ✓ Visualisation 3D Matplotlib

Accès sans installation

- Connecter le PC au hotspot HW-XXXXXXX
- Ouvrir un navigateur web
- Naviguer vers `http://192.168.149.1`
- Interface HTML5 complète, aucun logiciel requis

Mode : Contrôle à Distance

- 1 Vue caméra HD en direct
- 2 Zone de **contrôle souris** (arc de mouvement)
- 3 Boutons cartésiens $\pm X$, $\pm Y$, $\pm Z$
- 4 Sliders par **ID de servomoteur** (1–4)
- 5 Contrôle effecteur : ventouse / pince

Mode : Éditeur de Groupes d'Actions

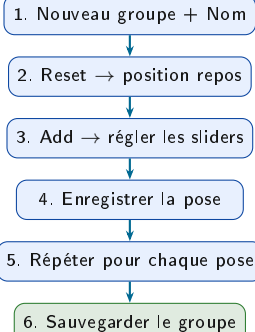
- 1 **Créer** un nouveau groupe d'actions
- 2 Saisir un **nom** identifiant pour le groupe
- 3 Cliquer **Reset** → position initiale
- 4 Cliquer **Add** → ajouter une pose
- 5 Ajuster les sliders (articulaire ou cartésien)
- 6 **Enregistrer** la pose dans la séquence
- 7 Répéter 4–6 pour toutes les poses requises
- 8 **Sauvegarder** le groupe d'actions

Deux modes de réglage des poses

- **Mode servo individuel**
Contrôle angle par angle (ID 1-4)
Précision maximale pour des gestes fins
- **Mode coordonnées cartésiennes**
Entrer (X, Y, Z) directement
Le MGI calcule les angles automatiquement

Durée de transition

Chaque pose a une **durée de transition** configurable (en ms) qui contrôle la vitesse de déplacement entre les étapes de la séquence.



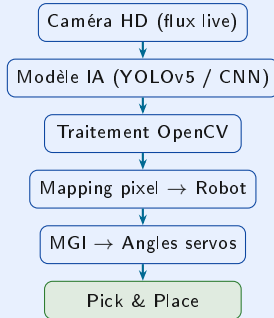
Applications visuelles (YOLOv5 / Lab + OpenCV)

- | | |
|-------------------------|-------------------------|
| ❶ Tri de déchets | YOLOv5 multiclasse |
| ❷ Séquencement d'objets | YOLOv5 lettres/chiffres |
| ❸ Tri de couleurs | Lab + contours OpenCV |
| ❹ Suivi d'objet | PID de centroïde |
| ❺ Calcul numérique | Reconnaissance chiffres |

Applications Deep Learning avancées

- | | |
|-----------------------------|------------------------|
| ❻ Empilement de blocs | Vision + planification |
| ❼ Suivi de visage | MTCNN + PID |
| ❽ Reconnaissance d'émotions | CNN facial |
| ❾ Reconnaissance de gestes | Keypoints mains |

Architecture commune des applications



Pipeline de traitement

- ➊ **Capture** : flux caméra HD temps réel
- ➋ **Inférence YOLOv5** : détecte et classe chaque objet (recyclable, biologique, autres déchets)
- ➌ **Sélection** : objet de **plus haute confiance** traité en premier
- ➍ **Pixel → Monde** : homographie sur plan table
- ➎ **Pick-and-Place** : saisie et dépôt dans la case correspondante au type de déchet

Commandes ROS

- `/waste_classification/enter` — initialise le nœud
- `/waste_classification/set_running` — lance la détection

YOLOv5 : avantage temps réel

Traitement **mono-étape** (one-shot) : la détection et la classification s'effectuent en **une seule passe** du réseau, compatible avec le GPU Maxwell du Jetson Nano.

Séquencement d'objets (variante)

- Détecte des cartes **alphabet / chiffres**
- Tri par ordre croissant :
min(taille boîte) → plus petit d'abord
- Script : `alphabetically_main.py`
- ROS : `/alphabetically/enter`

Réseau MTCNN (3 étages en cascade)

- 1 **P-Net** (Proposal Network)
Proposition rapide de régions candidates
- 2 **R-Net** (Refine Network)
Filtre et affine les boîtes candidates
- 3 **O-Net** (Output Network)
Boîte finale + **5 points clés** faciaux
(yeux, nez, coins de la bouche)

Asservissement PID de suivi

- Calcul du centre du visage (u_f, v_f)
- Erreur : $e_x = u_f - u_{\text{centre}}$, $e_y = v_f - v_{\text{centre}}$
- Correction PID $\Rightarrow$ déplacement angulaire $\Delta\theta_1$ (panoramique) et $\Delta\theta_2$ (tilt)
- Le bras suit le visage en temps réel

Prérequis matériel

La caméra doit être installée en position **horizontale** pour que le visage détecté soit correctement orienté par rapport aux axes de rotation du bras.

Commandes ROS

- `/object_tracking/enter`
— initialise le suivi
- `/object_tracking/set_running_face`
"data: true" — active le suivi facial

Pipeline de tri par couleur

- ❶ Conversion de l'image BGR → **Lab**
- ❷ Application des seuils ($L_{\min}$, $a_{\min}$, $b_{\min}$) calibrés avec l'outil de réglage
- ❸ `cv2.findContours` : extraction des formes
- ❹ **Plus grand contour** = objet cible sélectionné
- ❺ Calcul centroïde → homographie → coordonnées
- ❻ Dépôt dans la case R/V/B correspondante

Commandes ROS

- `/color_sorting/enter`
— initialise le nœud de tri
- `/color_sorting/set_target`
"color_name: 'red': is_enable: true"
— définit la couleur cible
- `/color_sorting/set_running` "data: true"
— lance le tri en continu

Couleurs supportées

Couleur	Axe Lab clé	Case
Rouge	<i>a</i> élevé	Rouge
Vert	<i>a</i> faible	Vert
Bleu	<i>b</i> faible	Bleu

Porte-Stylo (Pen Tool)

- **Montage** : 4× vis M3×8 + 2× vis M3×15
- Stylo inséré et serré dans le support imprimé 3D

Applications

- Tracer des formes géométriques (trajectoires cartésiennes — ligne, cercle)
- Dessiner des expressions d'émotions (IA)
- Écriture de texte (lettres, chiffres)

La précision millimétrique du PID rend le tracé exploitable pour l'écriture réelle.

Grande Pince — Classification GoogLeNet

- Modèle : **PyTorch + GoogLeNet** (Inception V1)
- **10 classes** : pomme, banane, poivron, carotte, citron, laitue, mangue, pomme de terre, piment rouge, rien
- 2 threads parallèles de dépose :
 - `move_1()` : fruit → panier fruits
 - `move_2()` : légume → panier légumes
- Script : `obj_classification.py`

Principe de la ventouse EM

- Fonctionne par **induction magnétique** (pièces ferromagnétiques uniquement)
- Activation / désactivation par signal GPIO
- Idéale pour : vis, écrous, rondelles métalliques

Positions de dépose calibrées

Objet	x	y	z
Vis	120 mm	0	140 mm
Écrou	-120 mm	0	140 mm

Pipeline YOLOv5 vis / écrous

- 1 Détection + classification **YOLOv5** (2 classes : vis, écrou)
- 2 Calcul centroïde pixel → coordonnées physiques
- 3 Déplacement vers la pièce détectée
- 4 Activation aimant → saisie magnétique
- 5 Tri séparé dans les deux zones dédiées

Interface ROS et script

- Script principal : `screw_nut.py`
- `/em_suction/enter`
— initialise le nœud EM
- `/em_suction/set_running`
— lance la détection continue

Administration Bas Niveau

- **Gravure OS** : Ubuntu 18.04 LTS via image JetPack SDK
- **Accès Headless** : démarrage automatique de **NoMachine (NX VNC)** pour bureau distant
- **Réseau** : configuration Wi-Fi
Mode AP (hotspot) et Mode Station (LAN)

Transferts et Commandes

- Protocole **SFTP** pour transmission de scripts Python
- Commandes Linux clés :

```
systemctl enable/start <service>
chmod +x script.py
dmesg | grep ttyTHS
```

Sauvegarde et Restauration (Leçon 10)

- Création d'images disque compressées de la microSD
- Commandes de sauvegarde :

```
dd if=/dev/mmcblk0 | gzip > backup.img.gz
```

- Restauration rapide après incident :

```
gunzip -c backup.img.gz | dd of=/dev/mmcblk0
```

- Garantit la **reprise en moins de 15 min**

Cerveau Bas Niveau (STM32)

- MCU dédié qui **décharge la Jetson Nano** de la gestion temps réel des moteurs
- Firmware embarqué pour le contrôle des servos et la gestion des actionneurs bas niveau

Bus Série Servomoteurs HTS-35H

- Liaison **half-duplex en cascade** à 115 200 bps
- Adressage par ID (1 à 4) — commandes individuelles
- Rétroaction temps réel :
 - Position angulaire
 - Température interne
 - Tension d'alimentation

Contrôle Avancé des Actionneurs

- Gestion simultanée de servos **PWM standard** (pour outils optionnels : grande pince, stylo)
- Pilotage de **moteurs DC de puissance**
- Activation du **buzzer** via MCU
- Gestion de la **pompe à vide** (GPIO)

Précautions physiques

Respecter les limites de courant par canal (max 3A). Ne pas inverser la polarité des connecteurs des servos sous peine de destruction du MCU.

Leçon 1 — Ventilateur Intelligent (MTCNN)

Caméra → détection faciale MTCNN → asservissement du bras → commande GPIO d'activation du ventilateur ciblé vers le visage détecté.

Leçon 2 — Scanner Ultrasons (I2C)

Lecture I2C de la distance (2–400 cm) et tracé cartographique en temps réel sur l'écran embarqué.

Leçon 3 — Émotions + Matrice LED TM1640

Classification CNN des expressions faciales. Tracé de motifs dynamiques (smiley / croix) par commande directe de bits sur la matrice LED 8×16 via le contrôleur TM1640.

Leçon 4 — Gestes + Tubes Numériques

- Inférence des points clés de la main (MediaPipe / réseau de keypoints)
- Compteur de doigts affiché sur l'afficheur à tubes numériques 7 segments

Leçon 5 — Contrôle Tactile (GPIO)

- **Interruption GPIO** sur bouton physique
- Démarre ou arrête les cycles du bras manipulateur sans passer par le logiciel PC
- Utile pour démonstrations en conditions réelles

Structure Matricielle (Numpy)

- Images = tenseurs de dimension 2 (niveaux de gris 8 bits)
ou dimension 3 (couleurs 24 bits : $H \times W \times 3$)
- **Ordre BGR** : OpenCV stocke les canaux dans l'ordre
Bleu-Vert-Rouge — attention lors des conversions
- Accès pixel : `image[y, x, c]`
- ROI (Region of Interest) : `image[y1:y2, x1:x2]`

Espaces Colorimétriques Comparés

Espace	Caractéristique
BGR	Sensible aux variations d'éclairage
HSV	Isole la teinte, mais saturation instable sur les bords/reflets
Lab	Sépare luminance (L) et couleur pure (a, b) — solution idéale

Lab est l'espace recommandé pour le seuillage de couleur robuste en conditions d'éclairage variables.

❶ Réduction du bruit (Gaussien) :

```
frame_gb = cv2.GaussianBlur(img, (5,5), 5)
```

❷ Conversion Lab :

```
frame_lab = cv2.cvtColor(frame_gb,  
                        cv2.COLOR_BGR2LAB)
```

❸ Seuillage binaire :

```
frame_mask = cv2.inRange(frame_lab,  
                        color_min, color_max)
```

❹ Opérations Morphologiques :

```
kernel = cv2.getStructuringElement(  
        cv2.MORPH_RECT, (3,3))  
eroded = cv2.erode(frame_mask, kernel)  
dilated = cv2.dilate(eroded, kernel)
```

Rôle des opérations morphologiques

- **Érosion** : élimine les petits bruits aux frontières du masque
- **Dilatation** : bouche les trous internes à l'objet détecté

Extraction des Contours

```
contours, _ = cv2.findContours(  
    dilated,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_NONE)
```

Recherche des frontières externes des zones blanches du masque binaire.

Filtrage Géométrique

```
contour_area = filter(  
    lambda c: cv2.contourArea(c) > 1200,  
    contours)
```

Rejet des bruits d'aire < **1200 pixels**.

Marquage Visuel à l'Écran

- Sélection du **plus grand contour** pour suivre l'objet principal
- Dessin de cercles de suivi :

```
cv2.circle(frame, center, r, color, 2)
```

- Réticule en croix :

```
cv2.line(frame, pt1, pt2, color, 2)
```

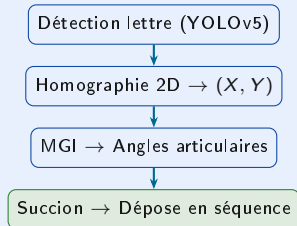
- Incrustation textuelle :

```
cv2.putText(frame, text, org,  
            font, scale, color, 2)
```

Reconnaissance de l'Alphabet (YOLOv5 TRT)

- Inférence en temps réel sur les **cartes de lettres** dispersées sur le tapis
- Variable `WORD_WANT` : définit le mot cible à épeler (ex : "DBRH")
- Modèle compilé TensorRT (.trt) pour inférence ultra-rapide sur GPU Maxwell

Exécution Cinématique



Algorithme de Décision Séquentielle

- ❶ **Lettre de référence** : `max()` sur la confiance de détection
- ❷ **Distance euclidienne** aux cibles de pose
- ❸ **Carte physique** la plus proche : `min(distance)`

Structure du Dataset facemask

- JPEGImages/ : images 640×480 pixels
- Annotations/ : fichiers XML annotés sous **labellmg**
(classes : facemask / nofacemask)
- ImageSets/ : listes train/val/test

Pipeline de Préparation

- 1 `python3 txt_gen.py` : répartition 80% Train / 15% Val / 5% Test
- 2 `python3 xml2yolo.py` : conversion PascalVOC XML → coordonnées normées YOLO
- 3 Création du fichier `facemask.yaml` (nc: 2)

Entraînement et Optimisation

```
# Entraînement PyTorch
python3 train.py --img-size 160 \
  --weights yolov5s.pt \
  --data data/facemask.yaml \
  --batch-size 4 --epochs 10

# Export ONNX
python3 models/export.py \
  --weights best.pt \
  --img-size 160 --batch-size 1

# Compilation TensorRT (GPU)
onnx2trt best.onnx -o best.trt
```

PyTorch → ONNX → TensorRT

Abonnement ROS & Traitement

- Souscription au topic caméra :

```
/usb_cam/image_rect_color
```

- Inférence directe via best.trt (YOLOv5 TensorRT)
- Dessin des boîtes englobantes :

```
cv2.rectangle(frame, pt1, pt2,  
              (0,0,255), 2)  
cv2.putText(frame, confidence, ...)
```

- Lancement : `roslaunch jetmax_demos face_mask_main.py`

Feedback Visuel — Matrice LED TM1640

- Contrôleur TM1640 via GPIO
Matrice LED 8×16 pixels

Détection	Affichage LED
Masque porté	Smiley souriant draw_smail()
Sans masque	Croix d'alerte draw_cross()

Résumé du pipeline

Caméra → YOLOv5 TRT → Classification → Feedback LED TM1640 en temps réel

Réalisations

- ❶ **Modélisation** : DH, MGD, MGI validés ($\varepsilon < 10^{-14}$ mm)
- ❷ **Matériel** : Jetson Nano + servos + caméra opérationnels
- ❸ **ROS** : Publisher, Subscriber et Service fonctionnels
- ❹ **Trajectoires** : PID précis ($e < 0.02$ mm)
- ❺ **Vision** : HSV + homographie + Pick-and-Place validés

Perspectives

- Déploiement **embarqué** sur le vrai JetMax
- Détecteur **YOLO v8** multi-classe
- Calibration intrinsèque complète (pinhole + distorsion)
- Planification sans collision avec **MoveIt** !
- Apprentissage par renforcement (RL)

Merci de votre attention

Questions et discussions ?